gr00t-arm

A desk robot arm that learns from demonstrations with NVIDIA GR00T

How it connects to GR00T, and how to build, wire, calibrate, train and run it

Yishane Kao

Arm: *Easy Robot Arm* by mobiobi (Printables 97370), CC BY-NC-SA, used unmodified

October 2026

Contents

I How the arm connects to GR00T **3**

1 The idea in one picture **3**

2 What GR00T is, in plain words **3**

3 Every GR00T concept, and where it lives on this arm **4**

4 The whole pipeline **5**

5 Presenting it in a humanoids talk **5**
5.1 A 3-minute script . . . 5
5.2 Live demo plan . . . 5
5.3 Questions you might get . . . 6

II Build and run instructions **6**

6 What you need **6**

7 Printing **6**

8 Laptop software setup **9**

9 Wiring **9**
9.1 Upload the firmware . . . 10
9.2 Check every servo before you fit it . . . 10

10 How the laptop talks to the Nano **10**
10.1 The USB cable is a serial port . . . 10
10.2 The messages . . . 11
10.3 What the Nano does with them . . . 11
10.4 Where the numbers come from . . . 11
10.5 The loop when GR00T drives . . . 11

11 Assembly **12**
11.1 Camera setup and framing check . . . 13

12 Calibration **13**

13 First moves **14**

14 Collecting demonstrations **14**

15 Training ACT on the laptop (free, optional but recommended) **15**
15.1 Learning from its own runs . . . 15

16 Fine-tuning and running GR00T in the cloud **16**
16.1 One-time accounts (you do these) . . . 16
16.2 Steps . . . 16

17 Simulation (MuJoCo): more demos and automatic testing **17**
17.1 How the simulation works . . . 18
17.2 Step 1: check the simulated camera against the real one . . . 19
17.3 Step 2: record simulated demos . . . 20
17.4 Step 3: train on real and simulated demos together . . . 20
17.5 Step 4: score models in the simulator . . . 21
17.6 Step 5: the real test . . . 21

18 Troubleshooting **22**

19 Where everything is **22**

Part I
How the arm connects to GR00T

1 The idea in one picture

GR00T is NVIDIA's *robot foundation model*: one neural network, pretrained on a huge amount of robot and human data, that turns **what a robot sees**, **what it is told** and **where its joints are** into **the next motions**. It was built for humanoids, but it is designed so a *new* body can be taught to it with a small amount of data. This arm is that new body: five $2 servos, a webcam for eyes, and 100–200 demonstrations. A ball and a block sit on the plate; you type "pick up the ball" or "pick up the block", and the arm drops that one in the cup. The words decide which object it goes for.

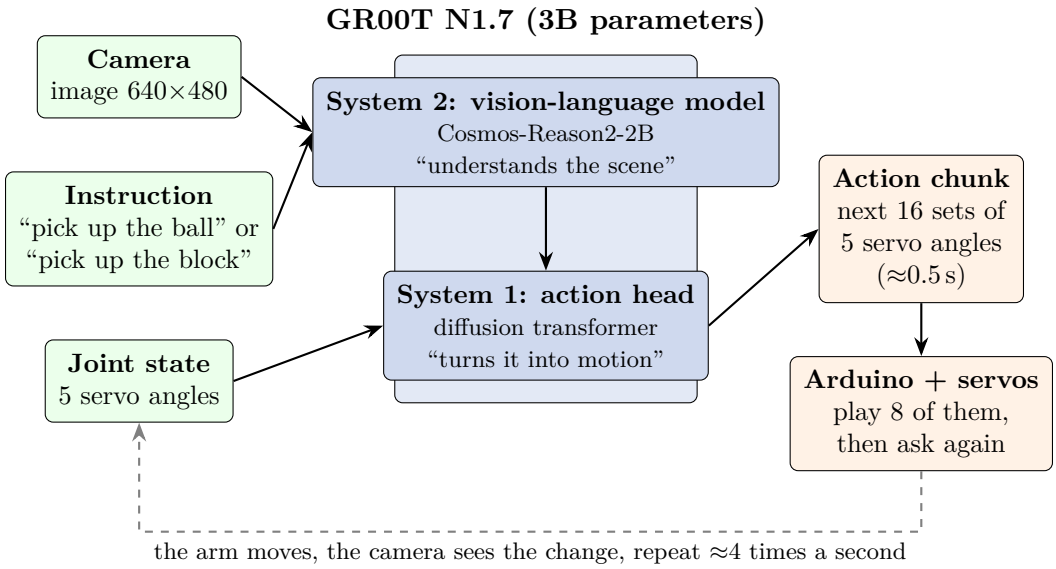


Figure 1: The control loop. Everything on the left is produced by this arm; the middle is GR00T; the right is sent back to the arm.

2 What GR00T is, in plain words

A foundation model for robots. Language models are pretrained on the internet and then adapted to a task. GR00T does the same for robot *motion*: it is pretrained on a mix of real robot recordings, simulated robots and videos of humans, then *post-trained* (fine-tuned) on a small set of demonstrations of a specific robot doing a specific task [1, 2].

Two systems, like fast and slow thinking. The **System 2** part is a vision-language model: it looks at the camera image and reads the instruction, and turns them into *embeddings*: long lists of numbers that capture what is in the scene and what was asked. Not words, and not positions. The **System 1** part is the *action head*, a diffusion transformer: it takes those numbers plus the robot's current joint angles and generates the motor commands itself. System 2 is the "what is going on and what should happen", System 1 is the "how do my joints get there".

It learns by copying, not by trial and error. Fine-tuning shows GR00T the recorded demonstrations and trains it to output what the demonstration did next in each situation (*imitation learning*).

There is no reward and no practising on the real arm. It never gets told where the ball is: it works that out from the camera image, which is the part the AI is for.

It outputs a short plan, not one move. The action head predicts an **action chunk**: the next 16 commands (about half a second). Predicting a chunk makes motions smooth and lets the model commit to a plan instead of dithering every frame. We play the first 8 and then ask again with a fresh image, so the arm keeps correcting itself.

How it generates actions: denoising. The action head starts from random numbers and refines them over a few steps (4 by default) into a sensible chunk, guided by what System 2 saw. This "flow matching" / diffusion trick is the same family of method image generators use, applied to joint angles instead of pixels.

One brain, many bodies. The big shared network is the same for every robot. What is specific to a body is small: how its joint state is read in and how actions are read out. GR00T has a reserved slot for robots it has never seen, `NEW_EMBODIMENT`. Our arm is registered there with 4 arm joints, 1 gripper and 1 camera.

3 Every GR00T concept, and where it lives on this arm

Table 1: Mapping GR00T's pieces onto the arm. File names are in the GitHub repo.

GR00T concept	On this arm	Where
Vision input	One USB webcam on a fixed mast looking down at the work area (key `front`). Fixed so every demo looks the same.	mast + webcam mount, `arm_io.py`
Language instruction	"pick up the ball" or "pick up the block", saved with each demo; typed in at run time	`recorder.py`, `run_policy.py`
Proprioceptive state	5 servo angles in degrees: base, shoulder, elbow, wrist (`single_arm`) and gripper (`gripper`)	`export_dataset.py`
Embodiment	`NEW_EMBODIMENT` with our modality config	`gr00t_arm_config.py`
Dataset format	LeRobot v2.1 + `meta/modality.json` (parquet tables + mp4 videos)	`export_dataset.py`
Action chunk	16 future commands; we execute 8 then re-query	`run_policy.py`
Relative vs absolute actions	Arm joints are predicted as *changes* from the current angles (generalises better); the gripper as an absolute open/close target	`gr00t_arm_config.py`
Post-training	Fine-tune on 100–200 demos on a rented cloud GPU, ≈3000 steps	`gr00t/finetune.sh`
Inference	Model runs on the cloud GPU as a server; the laptop sends image + state, gets actions back over ZeroMQ	`gr00t/serve.sh`, `policy_client.py`
Demonstrations	Scripted pick-and-place (auto-collected; random layout, ball or block about half each) or keyboard teleop	`autodemo.py`, `teleop.py`
Simulation data	A MuJoCo copy of the arm records extra demos at varied positions and scores models automatically; GR00T's pretraining also mixes in simulated robots	`sim.py`

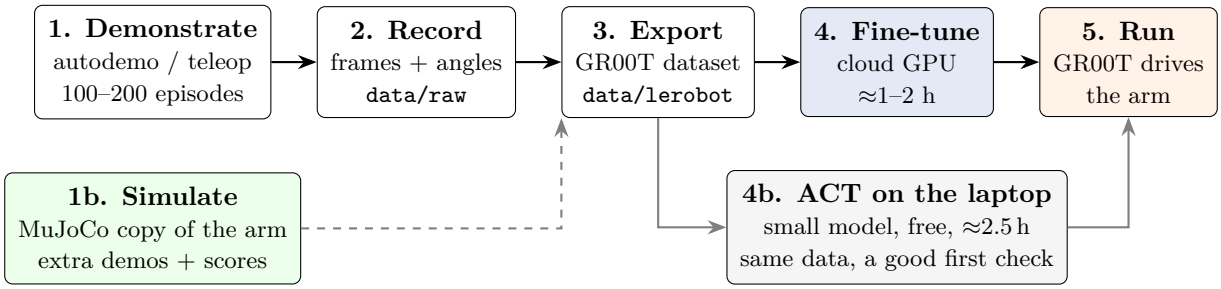


Figure 2: From demonstrations to a running policy. ACT [3] is a much smaller imitation-learning model that trains on the laptop; it checks the data before paying for a GPU. The simulation (Section 17) adds demos and scores models once the real demos exist.

4 The whole pipeline

5 Presenting it in a humanoids talk

5.1 A 3-minute script

What to say

1. **The problem.** "A humanoid has to turn what it sees and what it's told into motor commands for dozens of joints. Writing that by hand doesn't scale. NVIDIA's answer is GR00T, a foundation model for robots."
2. **Point at the arm.** "This is a $10 open-source arm, mobiobi's Easy Robot Arm: five hobby servos and a cheap webcam as the camera. GR00T doesn't care that it's small. The recipe is the same one used for humanoids."
3. **Inputs and outputs** (Fig. 1). "Three inputs: the camera image, an instruction in English, and the current joint angles. One output: a chunk of the next 16 joint commands, about half a second of motion."
4. **Two systems.** "Inside, a vision-language model figures out *what* to do, and a diffusion transformer turns that into *how* the joints move. Slow thinking and fast moving."
5. **New body.** "GR00T has never seen this arm. I registered it as a new embodiment: 4 joints, a gripper, one camera, and fine-tuned it on about 150 demonstrations, half picking the ball and half the block, each with its sentence. That took an hour or two on a rented GPU."
6. **Demo** (Section 5.2).
7. **Back to humanoids.** "A humanoid is the same loop with more joints, more cameras and much more data. Pretraining on many bodies is what lets each new body learn from only a handful of demonstrations."

5.2 Live demo plan

Pick the most ambitious option you have working, and keep the next one ready.

1. **GR00T live.** Start the cloud machine and `serve.sh` about 30 minutes before class, connect with `run_policy.py gr00t`. Needs internet. Put the ball and the block on two marked spots, ask the audience which one, and type it: `pick up the block`. When it's done it asks again, so swap the objects around and do the other one.

2. **ACT live (no internet).** Same pipeline, smaller model running on the laptop: `run_policy.py act`. Say clearly that this is the small model trained on the same data. ACT can't read words: it gets "ball" or "block" as a single number (0 or 1). That difference is a good point to make: GR00T understands the sentence, ACT only gets a switch.
3. **Scripted demo.** `autodemo.py --target block --spot C1 --other-spot A3` always works. Explain that this script is how the training demonstrations were collected, and that the learned policy never sees the spot names, only the camera image and the words.
4. **Video.** Screen-record a successful GR00T run the day before as a backup.

Show the dataset too: open one recorded `episode_000000.mp4` to make "demonstration data" concrete.

5.3 Questions you might get

Why not just program the pick-up? You can, and the scripted demo does. But that only works if the ball is exactly on a known spot. A learned policy reads the image, so the same approach scales to tasks nobody can script, like folding a shirt.

How much data? 100–200 demonstrations here, about half for each object. Pretraining is what makes that enough.

Does it really understand the words? Both objects are on the plate in every demo, so the only way to know which one to pick is the instruction. If it picks the right one after you swap their places, it's using the words and the image together.

Why chunks of actions? Smoother motion, fewer model calls, and the model commits to a plan. Replanning every 8 steps still lets it correct mistakes.

Does it generalise? Within reason: other spots on the plate, small lighting changes. Not to a new task it was never shown.

Did you train it in simulation? The demos came from the real arm; a simulated copy (MuJoCo) added demos with the objects at more positions and scored each model automatically. The training is the same either way: copying demonstrations.

Why the cloud? Fine-tuning GR00T needs a GPU with about 40 GB of memory and running it about 16 GB; the laptop has 8 GB. The cloud machine is only switched on while it's used.

Part II
Build and run instructions

6 What you need

7 Printing

The arm is mobiobi's *Easy Robot Arm* [4], printed without changes from the author's files in `cad/stl_opensource/easy_robot_arm/`. The work cell (baseplate, camera mast, phone cradle, cup) is ours, in `cad/stl`, generated from `cad/arm.py`.

Not needed: `BaseLeg` (the drum stands in a ring on the baseplate instead), `Arm75mm`, `Arm90mm`, `gripperTrunk`, `offAxis_gTrunk`, `twistMount` and the two `28BYJ-48` stepper parts. The `00`–`06` files in `cad/stl` are an older arm design and aren't used.

Table 2: Parts. Nothing exotic; most of it you already have. Beyond these, the only possible cost is cloud GPU time for GR00T, which has free options (Section 16).

Part	Qty	Notes
SG90 micro servo, **180°** (with horns and screws)	5	base, shoulder, elbow, wrist, gripper. **Not** the 360° / "continuous" version, which looks the same but can't hold an angle (Section 9.2). MG90S (same size, metal gears) also fits. The arm's design has room for a 6th (wrist roll); we leave it out
Arduino Nano + USB cable	1	the cable must carry data, not only charge
Adjustable bench power supply	1	set to 5.5 V, current limit 3 A
Breadboard + male–male jumper wires	–	to share power between servos
Superglue (CA)	–	optional: only if a part slips on its horn or the gripper turns
Orange soft foam ball (40 mm)	1	squeezable PU/stress-ball foam: grips better and rolls less than a ping-pong ball. Keep it 40 mm (the grip height and finger opening assume it); a ping-pong ball or two glued `12_ball_half` also work
Printed block (25 mm cube)	1	`13_block`, in a colour that stands out from the ball and the plate (green, with an orange ball on a light grey plate). Avoid a white ball: it barely stands out from the light grey plate
USB webcam with monitor clip	1	ours: KitGody 1080p, 90° view. Plug and play. Its clip has a 1/4"-20 tripod hole underneath. A phone with a webcam app (Iriun, DroidCam, Phone Link) on `09_phone_cradle` also works
1/4"-20 screw, ≈3/8" (10 mm) long	1	holds the webcam on `14_webcam_mount`. Hardware or camera store. A 1/2" screw works with a washer under its head
PLA filament	a few hundred g	Bambu printer; one 1 kg spool is plenty
Optional: 470–1000 μF capacitor	1	across the servo power rail; smooths current spikes
Optional: small elastic bands	–	the author's tip: around the joints to damp vibration

Table 3: Print list. Arm parts: the author's settings, PLA, 0.2 mm layers, supports on, 10% gyroid infill. Work-cell parts: PLA, 0.2 mm, 3 walls, 20% infill, no supports.

File	Qty	What it is / how to print
The arm (`cad/stl_opensource/easy_robot_arm`)		
`Base`	1	drum; the base servo sits inside
`BaseRotator`	1	turntable on the base servo; holds the shoulder servo
`BaseCap`	1	screw-on ring that holds the turntable on the drum
`Arm60mm`	2	upper arm and forearm (each carries the next servo)
`offAxis_gTrunkUndr`	1	wrist piece; hangs the gripper below the wrist axis
`gripperMount`, `gripperCap`	1 each	hold the gripper servo; the cap clamps it to the wrist piece
`gripperOne`, `gripper_Two`	1 each	geared fingers (they open sideways)
`gripperPinOne`, `gripperPinTwo`	1 each	finger pivot pins
The work cell (`cad/stl`)		
`07_baseplate`	1	flat; 250×200 mm, the longest print (15% infill is fine)
`08_camera_mast`	1	standing up, 4 walls
`15_mast_extension`	1	goes between the mast's top and the webcam mount, lifting the camera 80 mm; lying on one of its slanted faces, no supports. Print a second `10_pin` for it
`09_phone_cradle`	1	only if you use a phone as the camera; back face down
`14_webcam_mount`	1	the big flat face (with the square socket) on the bed; no supports
`10_pin`	1	lying flat
`11_cup`	1	upright, black
`12_ball_half`	2	only if you have no 40 mm ball; open side down
`13_block`	1	the second object; contrasting colour, 15% infill

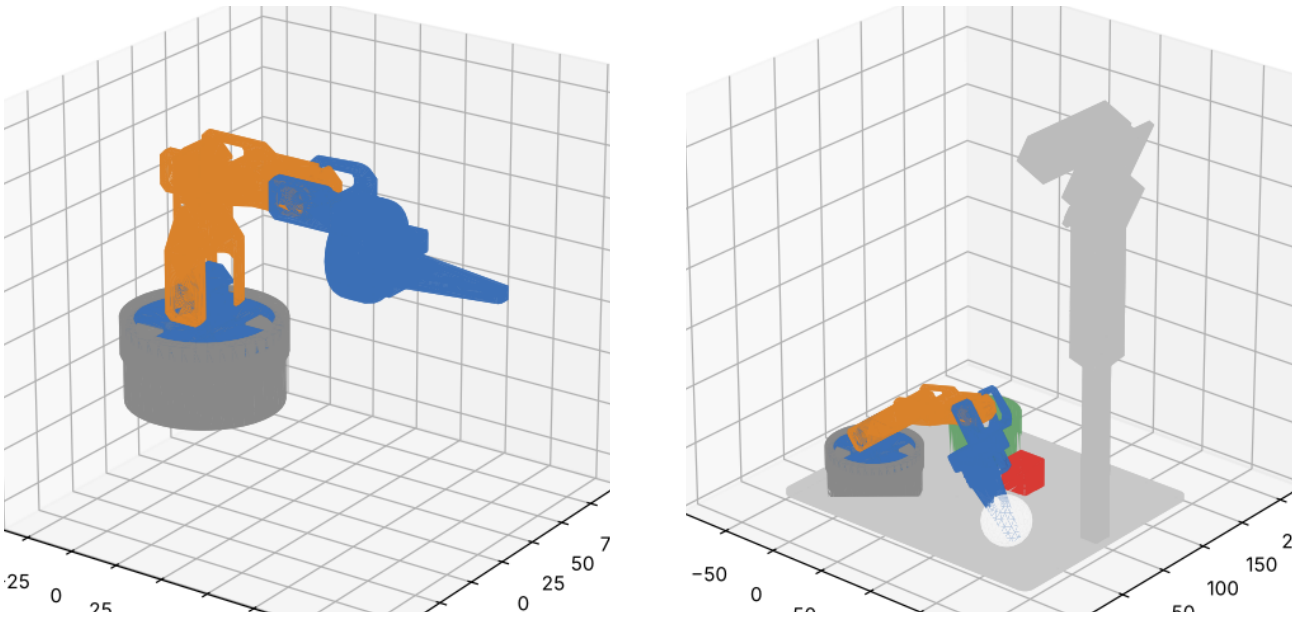


Figure 3: Left: the Easy Robot Arm in its home pose, built from the author's 3D model with two 60 mm links. Right: the work cell with the baseplate, cup and the webcam on its mast (with the 80 mm extension on top), the arm gripping the ball on C1 with the block on A3. `cad/assembly.py` checks the arm against itself and the work cell: every marked spot (ball and block), the lift above it and the drop over the cup are clear, and the gripper never touches the other object in any layout the demos use.

8 Laptop software setup

1. Install **uv** (Python manager) and **Arduino CLI** in PowerShell:

```
powershell -c "irm https://astral.sh/uv/install.ps1 | iex"
winget install ArduinoSA.CLI
```

2. Get the code and install the Python packages:

```
gh repo clone Yishane/gr00t-arm
cd gr00t-arm\software
uv sync
```

3. Plug in the USB webcam; it needs no driver. The laptop's own camera is usually index 0 and the webcam 1 (`--camera 1`). Using a phone instead: install **Iriun Webcam** or **DroidCam** on the phone and the PC (or Windows Phone Link on Android); it shows up as a normal webcam too.

9 Wiring

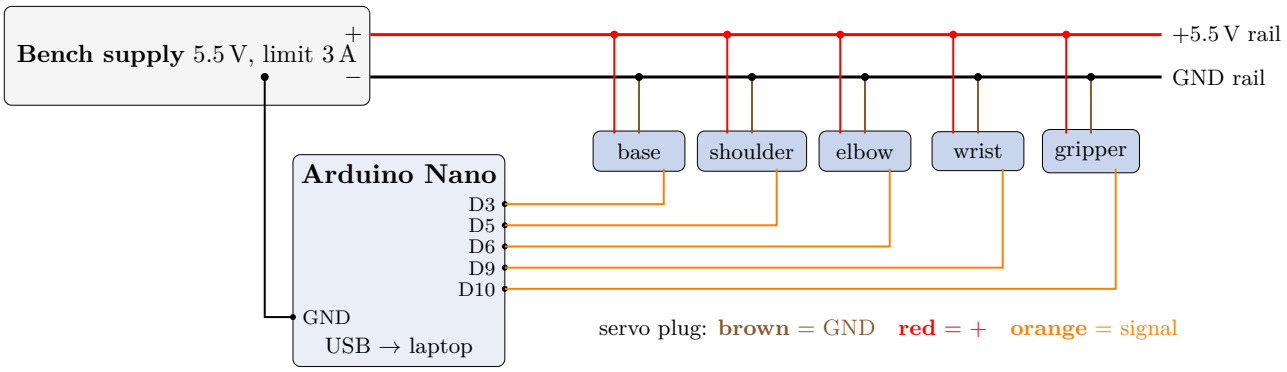


Figure 4: Wiring. Servo power comes from the bench supply, *never* from the Nano's 5V pin. The Nano's GND must connect to the supply's negative so the signals have a common reference. Orange signal wires go to D3, D5, D6, D9, D10 in the order base, shoulder, elbow, wrist, gripper.

1. Set the bench supply to **5.5 V** with the current limit at **3 A**, output off.
2. Bench supply + to the breadboard's red rail, − to the blue rail. (Optional: the capacitor across the two rails, its stripe/negative leg to the blue rail.)
3. Each servo: red to the red rail, brown to the blue rail (use jumpers into the servo's 3-pin plug), orange to its Nano pin from Fig. 4.
4. One jumper from a Nano GND pin to the blue rail.
5. Nano to the laptop by USB.

Why the separate supply

Five SG90s can pull over 2 A when they start moving together. USB gives 0.5 A: the Nano would reset and the arm would twitch. If the supply hits its current limit (the CC light comes on) something is jammed; turn it off and check.

9.1 Upload the firmware

Find the Nano's port in Device Manager → Ports (e.g. COM3), then:

```
cd gr00t-arm\firmware
.\upload.ps1 COM3
```

The script tries the "old bootloader" (most Nano clones) and then the new one. **Or with the Arduino IDE:** open `firmware/arm/arm.ino`, pick Tools → Board → Arduino Nano, Processor → ATmega328P (Old Bootloader) (if the upload fails, plain ATmega328P), Port → your COM port, then Upload. The firmware listens on USB serial for lines like `90,90,90,90,125` (five angles) and moves each servo smoothly toward its target.

9.2 Check every servo before you fit it

SG90s also come in a 360° "continuous" version that looks identical. It can't hold an angle: it treats 90 as "stop" and anything else as "keep spinning". Test each servo on its own, before it goes into the arm and with no horn on: send it 0, then 180 (in `calibrate.py`: `s 0 0`, then `s 0 180` with the servo on D3). A good servo turns to one end and *stops*, then to the other end and stops. If it keeps spinning, put it aside. Label each good servo with its joint as you plug it in.

10 How the laptop talks to the Nano

The laptop does all the thinking; the Nano only moves servos. About 30 times a second the laptop sends the Nano one line of text over USB with five angles in it, and the Nano moves the five servos toward them. GR00T never talks to the Nano: it runs on the cloud GPU, answers the laptop, and the laptop passes the answer on. With ACT the model runs on the laptop instead and everything below stays the same.

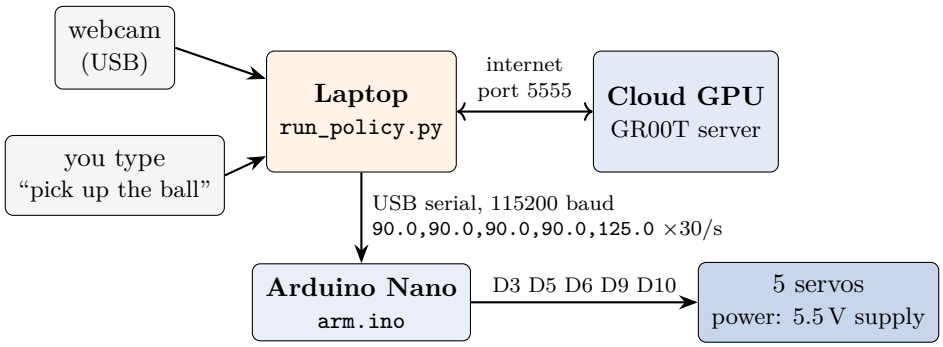


Figure 5: Who talks to whom. The Nano only ever sees five angles per line.

10.1 The USB cable is a serial port

- Plugged in, the Nano shows up as a COM port. The Python scripts find it by name ("Arduino", "CH340" or "USB"); pass `--port COM5` if they pick the wrong one.
- Speed is **115200 baud** on both sides (`BAUD` in `config.py`, `Serial.begin` in `arm.ino`).
- Opening a port normally **resets the Nano** (servos twitch, then snap to 90°). The Python scripts open it with the reset line (DTR) held off so it keeps running, ask `?` until it answers, and set

the speed back to 180°/s. When the Nano does start (power-up, reset button, Arduino Serial Monitor) it prints `READY` and moves every servo to 90°.

- Only one program can hold the port. Close the Arduino Serial Monitor before running Python.

10.2 The messages

Plain text, one command per line:

Laptop sends	The Nano
`90,120.5,45,90,125`	sets new target angles (degrees) for base, shoulder, elbow, wrist, gripper
`S 120`	sets the top servo speed to 120°/s (default 180)
`?`	replies `P a0,a1,a2,a3,a4`, the angles it is at now (for debugging; Python doesn't use it)

A line without exactly five numbers is ignored. You can type these into the Arduino Serial Monitor (115200 baud, "Newline" ending) to test the arm with no Python at all.

10.3 What the Nano does with them

1. Reads characters up to a newline, splits the line on commas.
2. Clamps each angle to 0–180° and stores it as that servo's *target*.
3. Every 10 ms moves each servo a small step toward its target, at most 180°/s, so a big jump in the commands becomes a smooth swing instead of a slam.
4. Turns each angle into a pulse (0° = 500 µs, 180° = 2400 µs) on that servo's signal pin.

The Nano knows nothing about kinematics, the camera or GR00T. It also can't tell where the servos really are: SG90s don't report their position, so it only remembers what it sent.

10.4 Where the numbers come from

The numbers on the wire are always *servo* angles (0–180°), not joint angles.

- `teleop.py` and `autodemo.py` think in gripper positions (x, y, z). `kinematics.py` turns a position into joint angles, then into servo angles using `calibration.json` (each servo's direction and zero).
- `run_policy.py`: the model was trained on servo angles, so its answer already *is* servo angles. The laptop clips each to 0–180° and sends it on.

10.5 The loop when GR00T drives

1. Grab a 640×480 photo from the webcam.
2. Package the photo, the five angles last sent, and your sentence.
3. Send it to the GR00T server (`policy_client.py`: a ZeroMQ request on port 5555 through the SSH tunnel). GR00T replies with **16** future steps of five angles each.
4. Send the first **8** steps to the Nano, one line every 1/30 s (about 0.27 s).
5. Repeat with a new photo, for 20 s (`--seconds`) or until Ctrl+C, then go home.

Steps 1–3 for the next chunk run *while* step 4 is still playing: once what is left to play takes about as long as the last call did (∼0.28 s to the cloud), the next request goes out in the background. The new chunk skips the steps planned for the time the call was out, so the arm doesn't stop between chunks; `--no-pipeline` turns this off. The "state" GR00T sees is the angles last *sent*, not measured ones, since the servos can't report back. The recordings were made the same way (every 1/30 s `recorder.py` saves the photo and the five angles just sent), so the model gets the same kind of state it learned from.

Power is a separate path

The USB cable only powers the Nano's chip and carries data. The servos run from the bench supply, and the supply's ground must be wired to the Nano's GND (Fig. 4). Without the common ground the signal pulses have no shared reference and the servos jitter or don't move.

11 Assembly

The author gives no written steps. The parts are made to need only what comes with the SG90s (the horns and screws), and every servo slides into a pocket so it can be swapped. Look at the photos on the Printables page [4] for how each piece sits.

The one rule

Every servo must be at **90 °** when the part it drives goes on, with that part in the **home pose**: arm facing forward, upper arm straight up, forearm level and pointing forward, gripper level and in line with the forearm, fingers closed. The software assumes this; get it right and calibration is only small trims.

1. **Center all servos.** Wire all five servos with nothing attached, power on, and run

   ```
   cd gr00t-arm\software
   uv run calibrate.py
   > center
   ```

   All five turn to 90 °. Leave them powered while fitting parts, and don't turn a shaft by hand.

2. **Base.** Base servo into the drum (`Base`), shaft up. Put the turntable (`BaseRotator`) on its horn so the arm will swing forward and back. Screw `BaseCap` over it onto the drum. Leave the `BaseLeg` feet off.

3. **Shoulder.** Shoulder servo into the turntable. The first `Arm60mm` (upper arm) onto its horn, pointing *straight up*.

4. **Elbow.** Elbow servo into the top of the upper arm. The second `Arm60mm` (forearm) onto its horn, pointing *forward, level*.

5. **Wrist.** Wrist servo into the end of the forearm. The wrist piece (`offAxis_gTrunkUndr`) onto its horn so the gripper will hang *below* the forearm, pointing forward and level.

6. **Gripper.** Gripper servo into `gripperMount`; the fingers on their pins with the gears meshed and the fingers *closed*; then clamp `gripperMount` to the wrist piece with `gripperCap`.

7. **No wrist-roll servo.** The design has room for one between the wrist piece and the gripper; we leave it out so the software stays at 5 joints. If the gripper turns on the wrist piece, put a drop of superglue between the two discs.

8. **Work cell.** Stand the drum in the ring on the baseplate (spots A1–C3 in front, the cup ring to the left) and fix it with double-sided tape or a few dots of hot glue, so the base servo turns the arm and not the drum. Push the mast into its socket, a pin into the mast top, the extension (`15_mast_extension`) onto that pin, a second pin into the extension's top, and `14_webcam_mount` onto that pin, its plate on the far side from the arm (or, with a phone, the phone cradle onto the pin). Clamp or tape the baseplate's back edge to the desk.
9. **Camera.** Fold the webcam's clip shut (the back leg flat under the top bar), rest the folded clip on the mount's plate against the lip, lens facing the arm, and screw it on through the slot from behind. A zip tie through the side slots is an extra hold. Set the head's own hinge straight, then do Section 11.1. (Phone: landscape on the cradle, resting on the lip, rear camera near the lip.)
10. **Cables.** Leave slack at every joint so nothing pulls when the arm turns.

11.1 Camera setup and framing check

1. **Webcam:** plug it in; nothing to set up. `14_webcam_mount` on the mast and its extension holds it about 325 mm above the plate, looking 65° down. On the 3D model that keeps the fingers, all nine spots and the cup in view during every scripted pick, and the elbow and wrist too (from 80 mm lower, without the extension, they left the top of the picture in a quarter to a third of the frames). The plate sits in the lower half of the picture: tilt the head about 5–10° further down on its own ±20° hinge to centre it. Once the check below passes, put a strip of tape across the hinge so it can't move. **Phone:** in the webcam app pick the **rear** camera and, if it lets you choose, the **0.5× (ultra-wide)** lens. 1× was tight from the phone cradle before the extension (the fingers would leave the top of the picture in roughly a quarter of the frames). Any resolution of 640×480 or more is fine; the code scales it to 640×480.
2. Find the camera index and see the live picture:

   ```
   uv run teleop.py --camera 1
   ```

   The window shows what the camera sees. Wrong camera or "didn't open"? Try `--camera 0` or `2`.
3. **Framing check.** Press [H] (home), then drive the gripper with [W][A][S][D] and [F] down to the corner spots A1 and C3 and over the cup, and with [R] up to about $z = 95$ (shown at the bottom of the window). The *fingers*, all nine spots and the cup must stay in the picture. The elbow and upper arm going off the top is fine: the model gets those joints as angles anyway.
4. If the fingers leave the picture: tilt the webcam's head on its hinge (or use 0.5× on a phone), or tell Claude what is cut off so the mount can be changed and reprinted.
5. Once it looks right, don't touch it: same camera, same position, same lighting for every recording and for the live demo. If the app has focus or exposure lock, turn it on.

If the shoulder struggles

The author notes the shoulder servo works hardest. If it sags or buzzes, swap just that one for an MG90S (same size, metal gears, no code changes). Small elastic bands around the joints, as in the author's photos, damp vibration.

12 Calibration

Still in `calibrate.py`:

1. `home` — the arm should go to the home pose.
2. `test` — moves each joint a little and tells you which way it should go. If one goes the wrong way, type `flip <joint>` (0 base, 1 shoulder, 2 elbow, 3 wrist).
3. If a joint is a few degrees off in the home pose: `trim <joint> <degrees>`, e.g. `trim 1 -4`.
4. Gripper: `s 4 <deg>` to try angles. Find an angle that is well open, and one that squeezes the *block* firmly (it's the smaller object, so this also grips the ball); set them with `grip open 125` and `grip closed 85` (your numbers). The fingers open sideways; check with the block and then the ball held between them. If the servo buzzes hard on the ball, tell Claude: the ball can get its own closed angle.
5. `save`, then `quit`. This writes `software/calibration.json`.

Check the tool point

The arm's lengths in `software/config.py` were measured from the author's 3D model, not from your printed arm. In Section 13, if the fingers close a little in front of, behind, above or below the ball, tell Claude how far off it is; `L_TOOL` (along the gripper) and `TOOL_UP` (how far the fingers hang below the wrist) get a small trim.

13 First moves

```
uv run autodemo.py --spot B2 --camera 1
```

Put the ball on spot B2 first. The arm should pick it up and drop it in the cup. Then the block, with the ball on another spot:

```
uv run autodemo.py --target block --spot C1 --other-spot A3 --camera 1
```

Turn the block so one face points at the arm's base: the fingers close sideways. Then try driving it yourself:

```
uv run teleop.py --camera 1
```

Click the window, then: W/S forward/back, A/D left/right, R/F up/down, Q/E tilt the gripper, Space open/close, H home, Esc quit. If it says "can't reach that", the target is outside the arm's reach or would hit the plate; it refuses rather than crashing. Add `--sim` to any script to try it without hardware.

14 Collecting demonstrations

1. **Automatic (recommended):**

   ```
   uv run autodemo.py --record 150 --camera 1
   ```

   Each time it tells you where to put the ball and the block and which one it will pick, e.g. `block on C1, ball on A3 -> "pick up the block"`. Place both, press Enter, and it picks and places

while recording, saving the instruction with the demo. It picks the ball or the block about half the time each. After each pick it asks `keep it? [Y/n]`: press Enter to keep the demo, or type `n` if it missed the grip, dropped the object or knocked the other one, and that demo is thrown away. `--record 150` keeps going until 150 demos are kept. Aim for 100–200 (about 20 seconds each, so 150 take under an hour; split it into sessions, it keeps numbering where it left off).

2. **By hand (optional, adds variety):** in `teleop.py`, T switches the instruction between "pick up the ball" and "pick up the block" (shown at the top of the window), Enter starts a recording, Enter again saves it, Backspace throws it away. Always pick the object the instruction names.

Good data

Keep the camera, lighting and cup exactly where they will be during the demo. Both objects are on the plate in every demo; the middle spot B2 isn't used because it's too close to the others. Place the objects a little off-center sometimes, with a block face towards the base. Answer `n` to every demo that went wrong: the model copies whatever it is shown, mistakes included. (A bad demo you only notice later: delete its folder from `data/raw/pick_ball`.)

Then export for GR00T:

```
uv run export_dataset.py
```

This writes `data/lerobot/pick_ball` (Table 1). GR00T's own data loader was used to check this format.

15 Training ACT on the laptop (free, optional but recommended)

```
cd software\act
uv sync
uv run export_act.py
uv run train_act.py --steps 20000
uv run ..\run_policy.py act --camera 1 ^
    --checkpoint ..\..\outputs\act\pick_ball\checkpoints\020000\pretrained_model
```

Training runs at about 2.3 steps per second on the RTX 5050 (about 2.5 hours for 20k steps). It asks for an instruction; type `pick up the ball` or `pick up the block` (or just `ball` / `block`). Each runs for 20 seconds (`--seconds`), then the arm goes home and it asks again; `q` quits. If ACT picks up the right object, the data is good and GR00T will learn from it too.

15.1 Learning from its own runs

The trained policy can record its own runs, and the good ones become extra training data:

```
uv run ..\run_policy.py act --camera 1 --record pick_ball ^
    --checkpoint ..\..\outputs\act\pick_ball\checkpoints\020000\pretrained_model
```

1. Type an instruction as usual. Every step the arm plays is saved: the camera frame, the five angles last sent (the state) and the five angles sent now (the action), exactly like a demo, tagged `"source": "policy"` in its `meta.json`. If the safety check cut a step short, the safe move the arm actually made is what gets saved.

2. Press Ctrl+C as soon as the object is in the cup, so the seconds after it aren't saved. The arm goes home and asks `keep it? [Y/n]`.
3. Keep a run *only* if it picked the right object cleanly. Training on its successes makes it better (it sees more object positions it handled well); training on its failures would teach it its own mistakes. Keep ball and block runs roughly even.

`--record pick_ball` puts the runs next to your demos, numbered after them, so the next export uses both; `--record` alone saves them to a separate `pick_ball_policy` set to look at first. Collect a batch (30–50 kept runs in one sitting), then export and train once, overnight. `train_act.py` won't overwrite an earlier model, so rename the old one first (and keep it to compare):

```
ren ..\..\outputs\act\pick_ball pick_ball_round1
uv run export_act.py
uv run train_act.py --steps 20000
```

The same works with GR00T: add `--record pick_ball` to `run_policy.py gr00t`, then export with `export_dataset.py` and fine-tune again. This only adds runs where the policy *already* succeeds, so it fills in positions near what it can do; for positions it fails at, it needs new demos (or the simulator, Section 17).

16 Fine-tuning and running GR00T in the cloud

16.1 One-time accounts (you do these)

1. **Hugging Face**: make a free account at https://huggingface.co, open https://huggingface.co/nvidia/Cosmos-Reason2-2B and request access (GR00T is built on it). Then create an access token (Settings → Access Tokens, "read").
2. **GPU time.** Fine-tuning needs a GPU with at least 40 GB (L40S, A100 or H100). Try free credits first. **Lightning AI**'s free plan gives monthly credits and a Studio you can SSH into, so the steps below work unchanged. **Modal**'s free plan gives about $30 a month, but it needs a small wrapper script first. Otherwise, rent a machine on a paid cloud such as RunPod or Lambda. Free 16 GB GPUs (Colab, Kaggle) are too small.

This is the only step that can cost money

Everything before this section is free. If you don't want to pay and there are no free credits, stop after ACT: it uses the same data and the same control loop, and it is a fine live demo (Section 5.2, option 2). Check the free plans' current terms on their pricing pages before signing up.

16.2 Steps

Do a dry run first: `docs/gr00t_dry_run.md` runs these same steps on a small fake dataset (`software/gr00t/dryrun`) in about an hour, so setup problems show up before the real data does.

1. Start a Linux GPU machine (Ubuntu, L40S/A100/H100, ≥100 GB disk). Note its SSH address and port.
2. Copy the GR00T scripts and the dataset up (from the `gr00t-arm` folder):

```
scp -P <port> -r software\gr00t data\lerobot\pick_ball root@<ip>:~/
```

3. Log in and set up (installs GR00T and downloads the base model, ≈20 min):

```
ssh -p <port> root@<ip>
export HF_TOKEN=hf_your_token
bash ~/gr00t/setup_cloud.sh
```

4. Fine-tune (about 1–2 hours; use `tmux` so it survives a disconnect):

```
tmux
bash ~/gr00t/finetune.sh ~/pick_ball
```

   At the end it plots predicted vs. recorded actions for episode 0 in `/tmp/open_loop_eval`. The lines should overlap. Flat predicted lines mean something is wrong with the data.

5. Serve the model:

```
bash ~/gr00t/serve.sh
```

6. On the laptop, open a tunnel in one terminal and run the arm in another:

```
ssh -p <port> -L 5555:localhost:5555 root@<ip>
uv run run_policy.py gr00t --host localhost --camera 1
```

   Type the instruction at the `instruction>` prompt, the same as for ACT.

7. **Stop the cloud machine when you're done**: it bills by the hour. You don't keep it running until the presentation: turn it on only to fine-tune, to test, and on demo day. Running the trained model needs far less GPU memory than training it, so a smaller, cheaper GPU (around 16 GB) is usually enough then. The laptop's RTX 5050 has 8 GB, which is probably too little, but it's worth one try once the model is trained.

Cost and time

A paid cloud GPU costs roughly $1–3 per hour. Setup plus one fine-tune plus testing is a few hours, so budget $10–20 for the first run, or use free credits if they cover it. Download the checkpoint folder (`~/gr00t_arm_ckpt`) if you want to reuse it later without retraining.

17 Simulation (MuJoCo): more demos and automatic testing

Once the real demos are recorded and ACT (and GR00T) have been trained on them, a simulated copy of the arm can add two things the real arm can't do quickly: hundreds of extra demos with the objects *anywhere* on the plate, and an automatic score for every trained model. It runs on the laptop, costs nothing, and the real arm can stay switched off.

What "training in simulation" means here

Training itself doesn't happen *inside* the simulator: ACT and GR00T still learn by copying recorded demos, on the laptop GPU or the cloud GPU, exactly as before. The simulator *makes* demos (the same kind of recording as on the real arm) and *tests* models (it knows where the ball ended up). This is also how GR00T itself was built: its pretraining mixes real robot data with simulated robots. Your real demos stay in the mix: the simulator never looks or behaves exactly like the real arm, and the real demos are what teach the model the real thing.

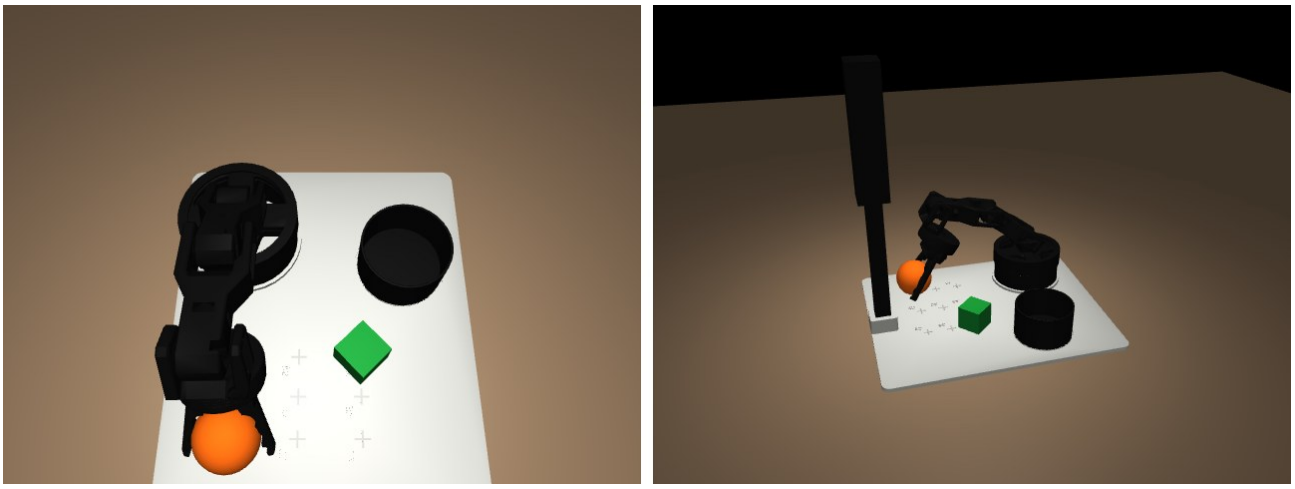


Figure 6: The simulation, rendered by `software/sim.py`. Left: what the simulated webcam sees as the arm reaches for the ball on C1 (this is the image the model gets). Right: the same moment from another angle. The arm's parts are the author's 3D model; the plate, cup, ball and block are ours.

17.1 How the simulation works

MuJoCo. A free physics engine (`pip install mujoco`, installed by `uv sync`). `sim.py` describes the work cell to it: the arm's moving parts, the plate, the cup, the ball and the block. MuJoCo works out how they move and collide 500 times a simulated second and draws what a camera would see.

The arm. Each part is the author's 3D model, on joints placed with the numbers in `config.py`: shoulder 54 mm above the plate, two 60 mm links, tool point 86.5 mm along the gripper and 24.75 mm below the wrist. So the simulated gripper is where `kinematics.py` says it is. (The parts were converted once, by `cad/sim_meshes.py`, into the files in `software/sim_assets`.)

Same commands as the real arm. Scripts send the simulated arm the same five servo angles they send the Nano. `sim.py` turns them into joint angles with *your* `calibration.json` (each servo's direction and zero), the same way `kinematics.py` does. So a recording made in the simulator has the same numbers as one made on the real arm, and the two can be trained on together. (Run the simulator on the laptop that has the calibration file.)

The servos. Like the firmware, each servo's target moves at most 180°/s, in 10 ms steps (Section 10). A simulated SG90 then follows it with limited strength (0.18 N m, an SG90's stall torque), so the arm sags a few millimetres under its own weight, as the real one does (about 4 mm at the tool in the home pose).

The gripper. The two geared fingers turn on their pins, mirrored, as the gears make them. Your calibrated "open" angle opens each finger 30°; your "closed" angle aims for 6°, a little narrower than

the block, so the fingers stop on the object and squeeze it with the servo's limited force (on the block at about 12°, on the ball at 13–16°), like the real calibration.

The objects. A 40 mm ball and a 25 mm block (6 g each), with grip and some rolling resistance like the foam ball. The block is turned with a face toward the base, as you place it. The arm collides with the plate, the cup, the mast and the objects, and the gripper with the base drum; the arm's parts don't collide with each other (the joint limits already stop that on the real arm).

The camera. A 640×480 picture from where `14_webcam_mount` holds the webcam, 65° down, seeing about 66×52°. The camera's exact place and view are estimates until you check them against a real photo (below).

A little different every time. For each new layout the light (direction, brightness), the colours (plate, ball, block, desk) and the camera (a few mm, 1–2°) change a little. A model trained on that can't rely on one exact look, which helps it cope with the real camera too.

Only layouts the arm can really do. Before it uses a layout, the simulator checks that both objects lie on the plate clear of the drum, cup and mast, at least 60 mm apart, and inside the camera's picture; then it tries the scripted pick in a hidden second copy of the world (physics only, about 0.1 s). Only if that pick works is the layout used. So a missed pick in a score is the model's fault, not an impossible layout.

It knows the result. After a pick it checks where the objects are: `ok` (the named object in the cup, the other one where it was), `knocked` (right object, but the other moved more than 15 mm), `wrong object`, or `missed`. Only `ok` counts as a success; no one has to answer `keep it?`.

Two clocks. For `teleop.py` and `run_policy.py` the physics keeps pace with the real clock, so a slow model call delays the arm exactly as on the real arm. For `autodemo.py` nothing has to wait, so the simulation moves 1/30 s per command and runs as fast as the laptop can draw the pictures.

Every script that takes `--sim` also takes `--mujoco`. They are different: `--sim` is the old no-physics stand-in (the arm only remembers what it was told and the "camera" is a moving square; the tests and the GR00T dry run use it), `--mujoco` is the simulation described here.

17.2 Step 1: check the simulated camera against the real one

The simulated pictures should look like the webcam's, or a model can't carry what it learns in one over to the other. Take one real photo with the arm at home, the ball on C1 and the block on A3 (a one-second teleop recording, kept out of your demos by its own dataset name):

```
uv run teleop.py --camera 1 --dataset camcheck
```

Press [H], then [Enter], wait a second, [Enter] again, [Esc]. Then:

```
uv run sim.py --compare ..\data\raw\camcheck\episode_000000\frames\000000.jpg
```

This writes `data/sim_compare.png`: the real photo and the simulated view side by side, and the two blended on top of each other below. If the plate, spots and cup don't line up, send the picture to Claude: the camera's position, tilt and field of view are three numbers at the top of `sim.py` (`CAMERA_POS`, `CAMERA_DOWN`, `CAMERA_FOVY`). `uv run sim.py --viewer` opens MuJoCo's own 3D window to look around (drag to rotate, double-click a part to select it), and `uv run teleop.py --mujoco` drives the simulated arm with the keyboard.

17.3 Step 2: record simulated demos

```
uv run autodemo.py --mujoco --record 300 --anywhere
```

The same scripted pick as on the real arm, but the simulator places the objects itself, and there are two ways (Fig. 7):

- `--anywhere`: anywhere the arm can pick and the camera can see, between the spots, in the middle (B2, which your real demos never use) and outside the grid. This is what lets the trained model handle objects you put down anywhere on that area.
- `--offset 15`: on the marked spots like your demos, each moved up to 15 mm in a random direction. Closer to your real demos, but it leaves gaps between the rows and around B2.

It keeps only the `ok` picks (it prints the result of each) and goes on until 300 are kept. They go in `data/raw/pick_ball_mujoco`, never mixed with your real demos by accident, and each one's `meta.json` says `"sim": true`. If 20 in a row fail, it stops: the calibration or the gripper angles are wrong.

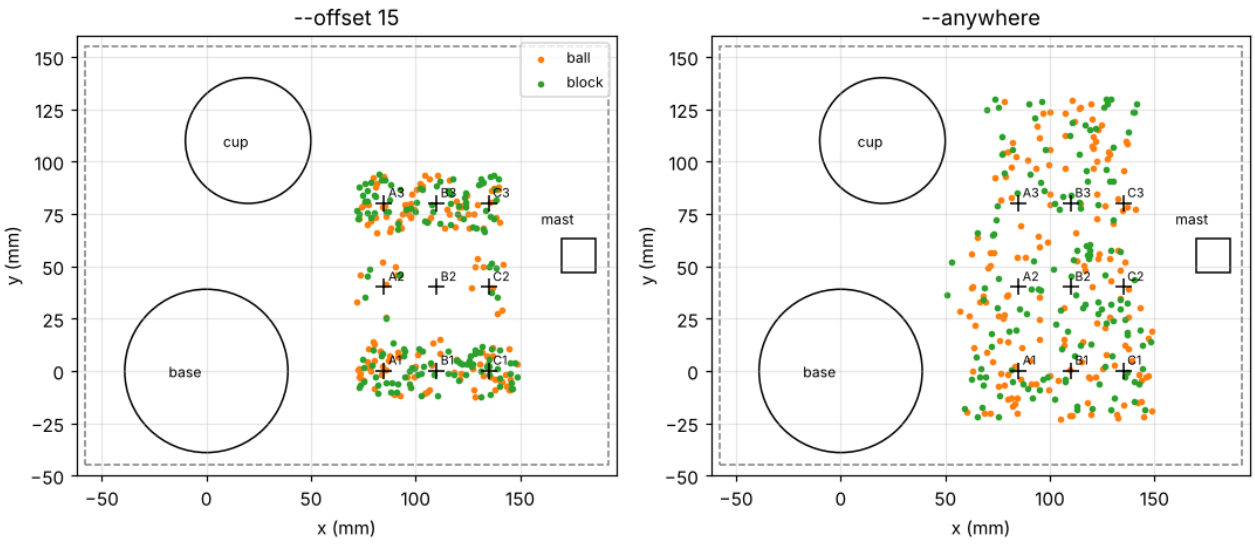


Figure 7: Where the simulator puts the object to pick, 300 layouts each. Left: `--offset 15` (round the spots). Right: `--anywhere` (everywhere the arm can pick and the camera can see). The empty strip on the right is where the open fingers would hit the mast's foot; further toward the camera an object leaves the bottom of the picture.

17.4 Step 3: train on real and simulated demos together

Export both sets into one dataset with `--with`, and give it its own name so the real-only model stays for comparison:

```
cd software\act
uv run export_act.py --with pick_ball_mujoco --out pick_ball_mix
uv run train_act.py --name pick_ball_mix --steps 20000
```

For GR00T, the same with `export_dataset.py`, then copy `data/lerobot/pick_ball_mix` up and fine-tune it as in Section 16:

```
uv run export_dataset.py --with pick_ball_mujoco --out ..\data\lerobot\pick_ball_mix
```

Start with about as many simulated demos as real ones (e.g. 150 real + 150–300 simulated). Too many simulated ones can make the model better at the simulator than at the real arm.

17.5 Step 4: score models in the simulator

```
uv run ..\run_policy.py act --mujoco --trials 20 ^
    --checkpoint ..\..\outputs\act\pick_ball_mix\checkpoints\020000\pretrained_model
```

Twenty runs, ball and block taking turns, each on a new random layout, each stopping as soon as an object lands in the cup. At the end it prints e.g. `ball:  9/10 ok (1 missed)`. Add `--anywhere` (or `--offset 15`) to test positions off the spots: that score says how well it copes with objects put down anywhere. Run the same command with the real-only model's checkpoint to compare the two; GR00T works too (`run_policy.py gr00t --mujoco --trials 20`, with the server and SSH tunnel running). Without `--trials` it asks for instructions like on the real arm and sets up a new layout for each.

Add `--record` and the `ok` runs are saved to `data/raw/pick_ball_mujoco`: this is the "learn from its own runs" loop of Section 15.1, with the simulator answering `keep it?`, so it can run unattended (e.g. `--trials 200 --record`).

17.6 Step 5: the real test

A simulator score is a quick way to compare models, not a promise. The model that counts is the one that works on the real arm: run the best one there (Section 15.1 commands, without `--mujoco`), with the same instructions and a few layouts on and off the spots, and compare it with the real-only model.

What the simulator gets wrong

- **Looks.** The pictures are drawn, not photographed: simpler light and surfaces, simpler shadows (on, as the simulated demos were recorded; a model has to be tested with the same look it was trained on, so `SIM_SHADOWS=0` is only for a model trained without them), no blue servo bodies or cables (only the printed parts are in the model), no camera noise or blur. That's why the real demos stay in the training mix, and why step 1 matters.
- **Servos.** SG90 dead band, gear slop and jitter aren't modelled, so the simulated arm moves more cleanly than the real one.
- **Geometry.** The simulated arm is exactly where `config.py` says. Corrections for the real arm's small errors (`SPOT_FIX` in `autodemo.py`) aren't used there.
- **Objects.** The foam ball is a firm sphere with soft contacts, not real foam.

18 Troubleshooting

Symptom	Fix
"No Arduino found"	Check the USB cable carries data; pass `--port COM5`.
Port opens but nothing moves	Supply off, no common ground, or the Serial Monitor is holding the port. Test by hand: Serial Monitor at 115200, type `90,90,90,90,125`, then `?` (Section 10).
Upload fails	Try the other bootloader (the script does both); close other programs using the port.
Servos twitch / Nano resets	Servo power must come from the supply; Nano GND tied to supply −; add the capacitor.
A joint moves the wrong way	`calibrate.py`: `test`, then `flip <joint>`.
Arm sags or buzzes holding pose	Normal for SG90s near their limit, mostly at the shoulder; keep the gripper light. An MG90S at the shoulder is a drop-in upgrade.
The drum turns instead of the arm	Fix the drum in its baseplate ring with tape or hot glue.
"Camera N didn't open"	Try `--camera 0`, `1`, `2`; is the webcam plugged in (or the phone app running)?
A servo spins non-stop	It's a 360° continuous servo; swap it for a 180° one (Section 9.2).
"can't reach that" in teleop	The target is out of reach or the gripper would hit the plate. Move back.
GR00T: `GatedRepoError`	Your HF account hasn't been granted Cosmos-Reason2-2B access yet, or `HF_TOKEN` is wrong.
`run_policy`: server not reachable	Is `serve.sh` running and the SSH tunnel open?
`run_policy`: "policy call failed"	One slow or failed GR00T reply; it retries. After 3 in a row it stops that run and asks again: check the SSH tunnel and `serve.sh`.
`run_policy`: "N limited"	The model asked for a pose past a joint limit or into the plate, so the arm only went as far as is safe (the gripper still opened or closed as asked). A few near the grip are normal: the demos grip only about 2 mm above the limit. Many all the time means more or better demos.
Policy runs but misses	More demos, more varied object positions, same camera position as in training.
Picks the wrong object	More demos (both objects about equally), and check the instructions saved in `meta.json` match what was picked.
`--mujoco`: "gladLoadGL error" or a black picture	The simulator couldn't draw: update the graphics driver. On a Linux machine without a screen it uses `MUJOCO_GL=egl` (needs `libegl1`).
"20 simulated picks in a row failed"	The simulator uses your `calibration.json`: check it with `calibrate.py test`, and that the gripper's open/closed angles are right.
"no reachable layout with `--offset`"	The offset puts the objects out of reach or out of view; use a smaller one (15 works), or `--anywhere`.
Good score in the simulator, poor on the real arm	The pictures differ too much: redo the camera check (Section 17), and keep more real demos in the mix.
`train_act.py`: "Output directory ... already exists"	An earlier model is there. Rename `outputs/act/pick_ball` (e.g. to `pick_ball_round1`) and run it again (Section 15.1).

19 Where everything is

`cad/` `arm.py` (work-cell parts), `assembly.py` (collision and reach checks on the author's model), `sim_meshes.py` (the simulation's meshes), `stl/`, `renders/`, and the arm itself (mobiobi's files, CC BY-NC-SA, with `ATTRIBUTION.md`) in `stl_opensource/easy_robot_arm/`

`firmware/` `arm/arm.ino`, `upload.ps1`

`software/` `config.py` (geometry, calibration), `kinematics.py` (+ tests), `calibrate.py`, `teleop.py`, `autodemo.py`, `recorder.py`, `export_dataset.py`, `policy_client.py`, `run_policy.py`, `sim.py` (the MuJoCo simulation) with its meshes in `sim_assets/`

software/gr00t/ `gr00t_arm_config.py`, `setup_cloud.sh`, `finetune.sh`, `serve.sh`

software/act/ `export_act.py`, `train_act.py`

context.md running notes on the project's state

References

[1] NVIDIA et al., "GR00T N1: An Open Foundation Model for Generalist Humanoid Robots," arXiv:2503.14734, 2025.

[2] NVIDIA, "Isaac GR00T" (code, GR00T N1.7 model, fine-tuning guides), https://github.com/NVIDIA/Isaac-GR00T.

[3] T. Z. Zhao, V. Kumar, S. Levine, C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," RSS 2023, arXiv:2304.13705.

[4] mobiobi, "Easy Robot Arm (SG90 servos & now steppers too)," Printables model 97370, CC BY-NC-SA, https://www.printables.com/model/97370-easy-robot-arm-sg90-servos-now-steppers-too-update.

[5] Hugging Face, "LeRobot," https://github.com/huggingface/lerobot.